

Especificação de Requisitos

Disciplina: Engenharia de Requisitos

Professor: Cloves Rocha

Estudantes: (01153674) Sidney Cirino, (01705254) Pedro Valença, (01709352) Wagner Daniel, (01706468) Fabricio Batista, (01704791) Joalison Silva.

Sumário

1. Introdução.....	1
1.1 Motivação.....	1
1.2 O Problema Identificado.....	1
1.3 Sobre a Organização.....	2
1.3.1 API de Produtos.....	3
1.4 Convenções para Identificação dos Requisitos.....	3
1.5 Convenções para Identificação dos Casos de Uso.....	4
1.5.1 Estrutura dos casos de uso.....	4
1.5.2 Prioridades dos casos de uso.....	4
1.5.3 Descrição dos Atores.....	5
2. Requisitos Organizacionais.....	5
3. Requisitos Funcionais.....	6 - 9
4. Requisitos Funcionais.....	9
4.1 Consulta do status do sistema.....	9
4.2 Consulta de resumo de produtos.....	9
4.3 Geração de relatório de integridade dos dados.....	9
4.4 Simulação de erros nos dados.....	10
4.5 Utilização de cache para otimização de requisições.....	10
4.6 Implementação de fallback para falhas externos.....	10
5. Requisitos Não Funcionais.....	11
5.1 Desempenho do sistema.....	11
5.2 Confiabilidade.....	11
5.3 Escalabilidade.....	12
5.4 Manutenibilidade.....	12
5.5 Portabilidade.....	12
6. Conclusão.....	13
6.1 Relatório da Equipe.....	13

1. Introdução

Este documento apresenta o problema identificado e os requisitos da solução proposta. O sistema desenvolvido consiste em um middleware de integração responsável por consumir dados de uma API externa, validar as informações recebidas e gerar métricas agregadas.

O middleware foi desenvolvido utilizando Python e Flask, funcionando como uma camada intermediária entre aplicações clientes e uma API de produtos, garantindo maior confiabilidade e consistência no acesso aos dados.

1.1 Motivação

Sistemas que dependem de APIs externas podem enfrentar problemas como falhas de comunicação, lentidão ou inconsistências nos dados retornados. Essas situações podem comprometer o funcionamento das aplicações que utilizam essas informações.

Diante disso, surgiu a necessidade de desenvolver um middleware capaz de intermediar o acesso aos dados, realizando validação e processamento das informações para aumentar a confiabilidade do sistema.

1.2 O Problema Identificado

Aplicações que dependem diretamente de APIs externas podem enfrentar diversos problemas, tais como:

- Falhas temporárias na API;
- Dados inconsistentes ou inválidos;
- Lentidão na comunicação com o serviço externo;
- Interrupções no serviço.

Esses problemas podem causar falhas no funcionamento de sistemas que utilizam essas informações.

Além disso, quando os dados retornados pela API possuem inconsistências, como campos ausentes ou valores inválidos, torna-se necessário realizar uma **validação rigorosa das informações antes de utilizá-las**.

Diante desse cenário, foi identificada a necessidade de um sistema intermediário capaz de:

- Consumir dados da API externa;
- Validar os dados recebidos;
- Detectar inconsistências;
- Gerar relatórios de integridade;
- Implementar mecanismos de resiliência para lidar com falhas externas.

1.3 Sobre a Organização

O sistema desenvolvido neste projeto atua como um **middleware de transparência e processamento de dados**, sendo responsável por intermediar a comunicação entre aplicações clientes e uma API externa de produtos.

Sua principal função é garantir que os dados consumidos por outras aplicações sejam **válidos, confiáveis e processados**, além de oferecer mecanismos que aumentem a estabilidade da comunicação com o serviço externo.

O middleware também fornece endpoints próprios que permitem consultar dados agregados, relatórios de integridade e informações sobre o estado da aplicação.

1.3.1 API de Produtos

A API externa utilizada no projeto fornece informações sobre produtos disponíveis em um catálogo online. Esses dados incluem informações como:

- Nome do produto
- Categoria
- Preço
- Descrição
- Metadados relacionados ao produto

O middleware desenvolvido realiza o consumo desses dados e aplica processos de validação para garantir que as informações estejam de acordo com o formato esperado. Caso inconsistências sejam encontradas, elas são registradas em um **relatório de integridade**, permitindo identificar possíveis problemas nos dados fornecidos pela API.

1.4 Convenções para Identificação dos Requisitos

Para facilitar a organização deste documento, os requisitos do sistema são apresentados de forma numerada e estruturada ao longo das seções.

Os requisitos descrevem as funcionalidades que o sistema deve oferecer, bem como características relacionadas ao seu funcionamento, como desempenho, confiabilidade e qualidade da aplicação.

A numeração utilizada permite identificar e localizar cada requisito com mais facilidade dentro do documento, além de auxiliar na organização das funcionalidades que compõem o sistema.

1.5 Convenções para Identificação dos Casos de Uso

Os casos de uso apresentados neste documento também seguem uma organização numérica. Cada caso de uso descreve uma interação entre um ator (como um usuário ou aplicação cliente) e o sistema.

Esses casos de uso têm como objetivo demonstrar como as funcionalidades do sistema podem ser utilizadas na prática, apresentando as etapas necessárias para a realização de determinadas operações.

A organização numérica facilita a compreensão do funcionamento do sistema e permite relacionar os casos de uso com as funcionalidades descritas anteriormente no documento.

1.5.1 Estrutura dos casos de uso

Cada caso de uso possui a seguinte estrutura:

Atores: usuários ou sistemas que interagem com a aplicação.

Prioridade: nível de importância da funcionalidade para o sistema.

Entradas: dados fornecidos ao sistema.

Pré-condições: condições que devem ser atendidas antes da execução do caso de uso.

Fluxo de eventos: sequência de ações realizadas durante a execução do caso de uso.

Saídas: dados retornados pelo sistema.

Pós-condições: estado do sistema após a execução do caso de uso.

1.5.2 Prioridades dos casos de uso

Os casos de uso são classificados em três categorias:

Essencial: indispensável para o funcionamento do sistema.

Importante: melhora a experiência do sistema, mas não é crítico.

Desejável: pode ser implementado em versões futuras.

1.5.3 Descrição dos Atores

Os atores que interagem com o sistema são:

- Aplicações clientes que consomem os endpoints do middleware;
- Sistemas externos que utilizam os dados processados pelo middleware.

2. Requisitos Organizacionais

Os requisitos organizacionais definem objetivos relacionados ao funcionamento e à utilidade do sistema dentro de um ambiente tecnológico.

Entre os principais requisitos organizacionais estão:

- Fornecer uma camada intermediária confiável para consumo de APIs externas;
- Garantir maior estabilidade no acesso aos dados provenientes de serviços externos;
- Fornecer mecanismos de monitoramento e validação das informações recebidas;
- Melhorar a confiabilidade das aplicações que dependem de dados externos.

3. Casos de Uso

Identificação:	[UC01] Consulta de Informações do Sistema
Atores:	.Aplicação cliente
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável
Entradas:	Requisição HTTP para um endpoint do sistema.
Pré-condições:	1. O middleware deve estar em execução. 2. A aplicação cliente deve possuir acesso aos endpoints da aplicação.
Fluxo de eventos:	1. A aplicação cliente envia uma requisição para consultar informações do sistema. 2. O middleware recebe a requisição. 3. O sistema verifica se existem dados disponíveis em cache. 4. Caso existam dados em cache, eles são utilizados para responder a requisição. 5. Caso não existam dados em cache, o sistema realiza a comunicação com a API externa. 6. O middleware coleta informações necessárias para responder a requisição. 7. O sistema organiza os dados obtidos. 8. O middleware retorna as informações para a aplicação cliente.
Fluxo alternativo:	1. Caso a API externa esteja indisponível, o sistema detecta a falha. 2. O middleware verifica se existem dados armazenados em cache. 3. Caso existam dados válidos, o sistema retorna essas informações como fallback.
Saídas:	Informações relacionadas ao estado atual do sistema ou dados processados pelo middleware.
Pós-condições:	A resposta pode ser armazenada em cache para otimizar futuras requisições.

Identificação:	[UC02] Processamento e Análise de Dados de Produtos
Atores:	.Aplicação cliente
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável
Entradas:	Requisição para executar testes ou simulações no sistema.
Pré-condições:	1. O middleware deve estar em execução. 2. O modo de teste ou simulação deve estar habilitado.
Fluxo de eventos:	1. A aplicação cliente solicita a execução de um teste no sistema. 2. O middleware recebe a requisição. 3. O sistema simula possíveis inconsistências nos dados recebidos da API externa. 4. O middleware executa o processo de validação de dados. 5. O sistema identifica erros ou inconsistências simuladas. 6. Um relatório é gerado com os problemas encontrados. 7. O resultado do teste é retornado para a aplicação cliente.
Saídas:	Relatório contendo os resultados da validação e das inconsistências detectadas.
Pós-condições:	O sistema confirma o funcionamento dos mecanismos de validação e detecção de erros.

Identificação:	[UC03] Teste e Validação do Sistema
Atores:	.Aplicação cliente / desenvolvedor
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável
Entradas:	Requisição para consulta de dados ou relatórios relacionados aos produtos.
Pré-condições:	3. O middleware deve estar em execução. 4. A API externa de produtos deve estar configurada.
Fluxo de eventos:	8. A aplicação cliente solicita informações relacionadas aos produtos. 9. O middleware realiza a requisição para a API externa de produtos. 10. O sistema recebe os dados retornados pela API. 11. O middleware executa um processo de validação dos dados recebidos. 12. O sistema identifica possíveis inconsistências ou dados inválidos. 13. O middleware processa os dados válidos. 14. O sistema gera métricas agregadas e relatórios de integridade. 15. O resultado é retornado para a aplicação cliente.
Saídas:	Resumo estatístico dos produtos ou relatório de integridade dos dados.
Pós-condições:	O sistema registra possíveis inconsistências encontradas nos dados analisados.

4. Requisitos Funcionais

Este capítulo apresenta as funcionalidades que o sistema deve oferecer.

4.1 Consulta do status do sistema

Identificação:	[RF01] Consulta do status
Casos de Uso relacionados:	[UC01]
Descrição:	O sistema deve permitir que aplicações clientes consultem o status atual da aplicação, incluindo informações como tempo de execução, ambiente em que o sistema está sendo executado, número total de requisições e outros dados relacionados ao funcionamento do serviço.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

4.2 Consulta de resumo de produtos

Identificação:	[RF02] Consulta de resumo
Casos de Uso relacionados:	[UC02]
Descrição:	O sistema deve permitir a consulta de dados agregados sobre os produtos obtidos da API externa, incluindo métricas como média de preços, mediana e quantidade de produtos por categoria.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

4.3 Geração de relatório de integridade dos dados

Identificação:	[RF03] Geração de relatório
Casos de Uso relacionados:	[UC02], [UC03]
Descrição:	O sistema deve analisar os dados retornados pela API externa e identificar possíveis inconsistências ou erros, gerando um relatório que apresente os problemas encontrados nos dados.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

4.4 Simulação de erros nos dados

Identificação:	[RF04] Simulação de erros
Casos de Uso relacionados:	[UC03]
Descrição:	O sistema deve permitir a simulação de erros nos dados recebidos da API externa, possibilitando testar e verificar o funcionamento do mecanismo de validação e detecção de inconsistências.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

4.5 Utilização de cache para otimização de requisições

Identificação:	[RF05] Utilização de cache
Casos de Uso relacionados:	[UC01]
Descrição:	O sistema deve armazenar temporariamente respostas válidas da API externa, utilizando cache para reduzir o número de requisições ao serviço externo e melhorar o desempenho da aplicação.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

4.6 Implementação de fallback para falhas externas

Identificação:	[RF06] Implementação de fallback
Casos de Uso relacionados:	[UC01]
Descrição:	O sistema deve retornar a última resposta válida armazenada em cache quando a API externa estiver indisponível, garantindo maior continuidade no funcionamento do serviço.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

5. Requisitos Não Funcionais

Os requisitos não funcionais deste sistema descrevem características relacionadas ao seu funcionamento e qualidade, incluindo aspectos como desempenho, confiabilidade, escalabilidade, manutenção e portabilidade.

5.1 Desempenho do sistema

Identificação:	[NFR01] Desempenho do sistema
Descrição:	O sistema deve responder às requisições realizadas pelos clientes em tempo adequado, garantindo que o processamento das informações e a geração das métricas ocorram de forma eficiente. Para melhorar o desempenho, o sistema deve utilizar mecanismos de cache temporário, reduzindo o tempo de resposta e evitando chamadas desnecessárias à API externa.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

5.2 Confiabilidade

Identificação:	[NFR02] Confiabilidade
Descrição:	O sistema deve manter seu funcionamento mesmo diante de falhas ou indisponibilidade da API externa. Para isso, devem ser utilizados mecanismos de resiliência que permitam lidar com erros de comunicação e manter a continuidade do serviço. Entre esses mecanismos estão tentativas automáticas de reconexão, utilização de dados previamente armazenados e estratégias para evitar múltiplas tentativas de acesso quando o serviço externo estiver indisponível.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

5.3 Escalabilidade

Identificação:	[NFR03] Escalabilidade
Descrição:	O sistema deve permitir a expansão da aplicação, possibilitando a adição de novos endpoints, novas fontes de dados ou novos mecanismos de análise sem comprometer a estrutura existente.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

5.4 Manutenibilidade

Identificação:	[NFR04] Manutenibilidade
Descrição:	O código do sistema deve ser organizado de forma modular e estruturada, facilitando a manutenção, atualização e correção de possíveis erros. A utilização de boas práticas de desenvolvimento deve permitir futuras melhorias na aplicação.
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

5.5 Portabilidade

Identificação:	[NFR05] Portabilidade
Descrição:	O sistema deve ser capaz de executar em diferentes ambientes que suportem a linguagem Python, incluindo ambientes de desenvolvimento local e servidores de aplicação
Prioridade:	☐ Essencial ☐ Importante ☐ Desejável

6. Conclusão

Por meio deste documento de requisitos foi possível apresentar o problema a ser resolvido e a proposta de desenvolvimento do sistema. O documento descreveu as principais funcionalidades que o middleware deve oferecer, bem como os requisitos necessários para garantir o correto funcionamento da aplicação.

Também foram apresentados os requisitos funcionais, que descrevem os serviços que o sistema deve disponibilizar, e os requisitos não funcionais, que definem características relacionadas ao desempenho, confiabilidade e funcionamento geral do sistema.

6.1 Relatório da Equipe

Nome	Esforço da equipe (%)	Atribuição
Wagner	20%	Desenvolvedor
Joalison Silva	20%	QA
Fabricio Batista	20%	Desenvolvedor
Pedro Valença	20%	Desenvolvedor
Sidney Cirino	20%	Desenvolvedor